1.5em

Roll Number

0	0	0	0	0	0	0
1	1	1	1	1	1	1
2	2	2	2	2	2	2
3	3	3	3	3	3	3
4	4	4	4	4	4	4
5	5	5	5	5	5	5
6	6	6	6	6	6	6
7	7	7	7	7	7	7
8	8	8	8	8	8	8
9	9	9	9	9	9	9

The OMR Sheet will be computer checked. Fill the circles completely and dark enough for proper detection. Use Dark HB pencil or ballpen (black or blue) for marking. While erasing, erase properly.

Avoid Improper Marking

Partially Filled

Lightly filled

Analysis & Design of Algorithms 2026 — Quiz 5

Duration: 1 hour Total Marks: 20

Name: _____ **Roll No.:** _____

Problem 1. (10 points)

1. Consider an undirected weighted graph $G = (V, E)$ with w_e as edge weights for every edge $e \in E$. The weights are distinct real numbers. Consider *any* cycle C in the graph and let e be the edge with maximum weight among the edges in C. Prove formally that e *cannot* be part of the minimum spanning tree of G.

   Answer:

   Proof by Contradiction: Assume for the sake of contradiction that the edge $e = (u, v)$, which has the maximum weight in cycle C, is part of the MST T.

   If we remove e from T, it partitions the spanning tree into two disconnected components, say T_1 and T_2, with $u \in T_1$ and $v \in T_2$. Because e is part of the cycle C, there must be at least one other edge $e' = (x, y)$ in C that crosses the cut between T_1 and T_2 to close the cycle.

   Since edge weights are distinct and e is the heaviest edge in C, we know that $w(e') < w(e)$. Let us form a new tree $T' = (T \setminus \{e\}) \cup \{e'\}$. Because e' reconnects the two components T_1 and T_2, T' is a valid spanning tree. The total weight of this new tree is:

   $$w(T') = w(T) - w(e) + w(e') \tag{1}$$

   Since $w(e') < w(e)$, it follows that $w(T') < w(T)$. This directly contradicts the assumption that T is a Minimum Spanning Tree. Thus, the heaviest edge e in any cycle C cannot be part of the MST.

   Rubric:

   - **+2 Marks:** Correctly sets up a proof by contradiction (e.g., assumes the heaviest edge e is in the MST).
   - **+2 Marks:** Identifies that removing e creates a cut/disconnects the tree, and another edge e' from the cycle must cross this cut.
   - **+1 Mark:** Concludes that swapping e for e' creates a valid spanning tree with a strictly smaller weight, completing the contradiction.

   Alternate way of writing the same proof: Compare 2 configurations of the MST, one with e and one with e'; compare the two as in Eq.(1)

2. Give the high-level description of a polynomial time algorithm to find MST that can directly utilize the above fact. We need a precise description of the main idea behind the algorithm in 2 sentences (implementation details or pseudocode is not required) in English and a formal proof of correctness that uses part (a).

There are 3 possible solutions:

(I) **Algorithm description [Reverse-Delete]:** Sort all edges in descending order of their weights, and iterate through them one by one; if removing the current edge does not disconnect the graph (i.e., it is part of some cycle), delete it from the graph. Continue until no such edges can be removed, leaving only the MST.

Correctness: The algorithm maintains a connected graph and terminates only when removing any remaining edge would disconnect it. This guarantees the final output is a connected, acyclic graph—a valid spanning tree. Whenever the algorithm deletes an edge e, it is because its removal does not disconnect the graph, meaning e is part of some cycle C in the current graph. Since we process the edges in strictly descending order of weight, e is guaranteed to be the strictly heaviest edge in that cycle C. By the Cycle Property established in Part A, the heaviest edge in any cycle cannot belong to the true MST. Therefore, Since it outputs a valid spanning tree without ever discarding an actual MST edge, the result must be the Minimum Spanning Tree.

(II) **Algorithm description [Kruskal's]:** Sort all edges in the graph in ascending order of their weights and initialize an empty forest. Iterate through the sorted edges one by one, adding each edge to the forest if it connects two different components, but completely discarding the edge if adding it would form a cycle.

Correctness: When the algorithm (Kruskal's) evaluates an edge e and determines that adding it to the current forest would create a cycle C, the algorithm discards e. Because we pre-sorted all edges in strictly ascending order of their weights, every edge already present in the forest and therefore every other edge making up cycle C was processed before e. This guarantees that e is the strictly heaviest edge in the newly formed cycle C. By the Cycle Property established in Part a, the strictly maximum weight edge in any cycle cannot be part of the MST. Therefore, whenever Kruskal's algorithm discards an edge that forms a cycle, it is safely discarding an edge that is proven to not be in the MST. As it continues until the graph is fully connected without cycles, the resulting structure is exactly the Minimum Spanning Tree.

(III) **Algorithm description [Cycle detection]:** Let the input graph be $G = (V, E)$. Initialize a working graph $T = (V, E_T)$ where initially the edge set is $E_T = E$. Repeatedly run a standard graph traversal algorithm (like Depth-First Search or Breadth-First Search) on T to detect if any cycle C exists. If a cycle C is found, identify the edge $e \in C$ with the strictly maximum weight and remove it from E_T (i.e., $E_T \leftarrow E_T \setminus \{e\}$). Continue this process until the traversal confirms no cycles remain in E_T, at which point the graph $T = (V, E_T)$ is the Minimum Spanning Tree.

Correctness: The algorithm starts with the original connected graph $G = (V, E)$. It only ever deletes an edge e from E_T if that edge is part of a cycle. Removing an edge from a cycle never disconnects the set of vertices V. The algorithm terminates strictly when there are no cycles left in E_T. A connected, acyclic graph spanning all original vertices V is, by definition, a valid spanning tree and by using part a, it is a MST.

Rubric:

- **+2 Marks (Algorithm):**
  - +2 Marks: Clearly describes an algorithm that either systematically removes edges from cycles or builds a tree while preventing cycles. They must indicate a sorting mechanism (e.g., descending/ascending) OR an active search mechanism (e.g., DFS/BFS cycle detection).
  - +1 Mark: The algorithm is broadly correct but lacks a critical detail that guarantees polynomial time or correctness (e.g., says "find cycles and delete the heaviest edge" but forgets to mention how to find them, or forgets to specify that edges must be sorted).
  - 0 Marks: The algorithm is fundamentally flawed or does not produce an MST.
- **+2 Mark (Proof Link):**
  - +2: Explicitly argues that whenever their algorithm discards an edge, that specific edge is guaranteed to be the strictly heaviest edge in a cycle. They then explicitly state that, according to Part A, this makes it safe to discard.
  - +1: Mentions Part A or implies the connection, but the logical link is weak. (e.g., They say "By Part A this works," but fail to explain why their algorithm's discarded edge is actually the heaviest in its cycle).
  - 0 Marks: No attempt is made to use the cycle property from Part A in the proof.
- **+1 Marks (Validating the final MST):** States a valid termination condition that guarantees the final output is a MST (e.g., "It stops when no cycles remain, leaving a connected acyclic graph" or "It stops when all vertices are connected without cycles") and using part a we have MST.

Problem 2. (10 points) Let $G = (V, E)$ be a directed acyclic graph whose vertices have labels from some fixed alphabet, and let $A[1 \dots k]$ be a string over the same alphabet. Hence, any directed path in G has a *label*, which is a string obtained by concatenating the labels of its vertices. Describe an algorithm running in time $O(k \cdot |E| + |V|)$ that either finds a path in G whose label is A, or correctly reports that no such path exists. You must have realized that you will need to design a DP for this problem (there is no other way to solve it, so don't try anything else). You will need to write the usual details of a dp - (a) Precise subproblem definitions. (b) Recurrences on the basis of (a) along with base cases and the subproblem that will give the final answer. (c) Pseudocode on the basis of (a) and (b). (d) Runtime analysis.

Answer: Let $L(v)$ denote the label assigned to vertex v.

Subproblem: Let $DP : V \times \{1, \ldots, k\} \to \{\text{True}, \text{False}\}$ be defined such that $DP(v, i)$ evaluates the existence of a valid path starting from vertex v matching the suffix of string A from index i to k.

$$DP(v,i) = \begin{cases} \text{True} & \text{if } \exists \text{ path } (v_1, v_2, \ldots, v_{k-i+1}) \text{ in } G \text{ where } v_1 = v \\ & \text{and } L(v_j) = A[i+j-1] \text{ for all } 1 \le j \le k-i+1 \\ \text{False} & \text{otherwise} \end{cases}$$

Base Case, Recurrence and Final answer

1. Base Case (when $i = k$):

$$DP(v,k) = \begin{cases} \text{True} & \text{if } L(v) = A[k] \\ \text{False} & \text{if } L(v) \neq A[k] \end{cases}$$

2. Recurrence Relation (when $1 \le i < k$):

$$DP(v,i) = \begin{cases} \text{True} & \text{if } L(v) = A[i] \wedge (\exists u \in V \text{ such that } (v,u) \in E \wedge DP(u, i+1) = \text{True}) \\ \text{False} & \text{otherwise} \end{cases}$$

3. Final Subproblem Answer:

$$\text{Final Answer} = \begin{cases} \text{True} & \text{if } \exists v \in V \text{ such that } DP(v,1) = \text{True} \\ \text{False} & \text{otherwise} \end{cases}$$

Pseudocode:

ReconstructPath in the below can be considered a blackbox

Algorithm 1 FindStringPath($G = (V, E, L),\ A[1 \ldots k]$)

```
Initialize DP table:
for all v ∈ V do
    for i = 1 to k do
        DP(v, i) ← False
    end for
    if L(v) = A[k] then
        DP(v, k) ← True
    end if
end for

Fill DP table:
for i = k − 1 down to 1 do
    for all v ∈ V do
        if L(v) = A[i] then
            for all u ∈ adj(v) do
                if DP(u, i + 1) = True then
                    DP(v, i) ← True
                end if
            end for
        end if
    end for
end for

Check for valid starting vertex and reconstruct path:
for all v ∈ V do
    if DP(v, 1) = True then
        return ReconstructPath(v, A, DP)
    end if
end for
return False
```

Runtime Analysis:

1. **Initialization:** The DP table is initialized using two nested loops over V and $[1 \dots k]$. Thus, the time complexity is $O(k|V|)$
2. **Filling the DP Table:** For each $i = k-1$ down to 1, we iterate over all vertices $v \in V$ and their adjacency lists. Hence, each iteration over i takes $O(|V| + |E|)$ time. Since this is repeated for k values $O\big(k(|V| + |E|)\big)$
3. **Path Reconstruction:** Reconstructing the path involves traversing edges (e.g., via BFS or guided traversal), which takes $O(|V| + |E|)$

Total Time Complexity: Combining all components:

$$O(k|V|) + O\big(k(|V| + |E|)\big) + O(|V| + |E|)$$

$$= O\big(k|V| + k(|V| + |E|) + (|V| + |E|)\big)$$

$$= O\big(k(|V| + |E|)\big)$$

Final Complexity:

$$\boxed{O\big(k(|V| + |E|)\big)}$$

Rubric:

- **(a) Subproblem Definition (2 Marks)**
  - **+2 Marks:** Clearly defines a DP state that tracks both the current vertex v and the current position i in string A. *(Award 1 mark if they only track the vertex or only the string index).*
- **(b) Recurrence & Base Cases (3 Marks)**
  - **+1 Mark:** Correct base case for the end of the string.
  - **+1 Mark:** Correctly checks if the current vertex label matches $A[i]$.
  - **+1 Mark:** Correctly transitions to a neighboring vertex u and index $i + 1$.
- **(c) Pseudocode (3 Marks)**
  - **+1 Mark:** Correct initialisation of DP table
  - **+1.5 Mark:** Correctly coding the recurrence
  - **+0.5 Mark:** Returning False if path doesn't exist, or returning the path if it exists.
- **(d) Runtime Analysis (2 Marks)**
  - **+0.5 Mark:** Correct runtime analysis of filling the DP table
  - **+1 Mark:** Correct runtime analysis of the recurrence
  - **+0.5 Mark:** Correct runtime analysis of path reconstruction
- **Alternate Approach:** The above solution processes the string in reverse $A[k]$ *to* $A[1]$, but processing it from $A[1]$ will also be valid provided the base case, recurrence and final outputs are processed accordingly. The marks division remains the same in both cases.
- **Note:** Processing the vertices in any order is valid, e.g applying topological sort to fill the DP table, while it does not provide any advantage, is not incorrect.